

Современные методы анализа данных и машинного обучения

Тема 1. Лекция 1

Введение в Python и библиотеки для анализа данных

Юрий Саночкин

ysanochkin@hse.ru

НИУ ВШЭ, 2026

Особенности Python

- Язык программирования
- Для чего он используется? Какие задачи решаются с помощью Python?

Особенности Python

- Язык программирования
- Для чего он используется? Какие задачи решаются с помощью Python?
- Относительно медленный язык, в сравнении с С-подобными
- Не подходит для высоконагруженных систем
- Не очень удобен для разработки, хотя используется
- Самый распространенный язык для анализа данных, аналитики, машинного обучения

Что особенного нам понадобится для анализа данных?

- Как вы думаете, что должен «уметь» язык для задач анализа данных?

Что особенного нам понадобится для анализа данных?

- Как вы думаете, что должен «уметь» язык для задач анализа данных?
- Можно назвать много важных требований
- Самое ключевое – уметь быстро и эффективно работать с большим объемом данных
- Ну, и визуализировать найденные зависимости

Что особенного нам понадобится для анализа данных?

- Как вы думаете, что должен «уметь» язык для задач анализа данных?
- Можно назвать много важных требований
- Самое ключевое – уметь быстро и эффективно работать с большим объемом данных
- Ну, и визуализировать найденные зависимости
- Сам Python не вполне умеет это делать
- Зато специальные библиотеки – умеют!

Библиотеки в Python

- Существует огромное количество библиотек, в том числе предназначенных для разного рода анализа данных
- Самые ключевые и распространенные, знания которых необходимы любому аналитику:
 - NumPy – библиотека для эффективной работы с матрицами, векторами и другими математическими объектами (очень быстрая)
 - Pandas – библиотека для работы с данными, представленными в виде матрицы
 - Matplotlib – библиотека для визуализации данных

Импорт библиотек в Python

- Какие есть способы импортировать библиотеку?

Импорт библиотек в Python

- Какие есть способы реализовать функционал из библиотеки?

1) Просто импортировать библиотеку

```
[1] import math  
  
sinus = math.sin(math.pi / 2)  
print(sinus)
```

```
1.0
```

Импорт библиотек в Python

- Какие есть способы реализовать функционал из библиотеки?

2) Импортировать нужные функции из библиотеки

```
[2] from math import sin, pi
```

```
    sinus = sin(pi / 2)  
    print(sinus)
```

```
1.0
```

Импорт библиотек в Python

- Какие есть способы реализовать функционал из библиотеки?

3) Импортировать всё из библиотеки (плохой вариант)

```
[3] from math import *
```

```
sinus = sin(pi / 2)  
print(sinus)
```

```
1.0
```

Почему этот вариант плохой?

Импорт библиотек в Python

- Основные библиотеки для анализа данных обычно импортируют первым способом, используя «общепринятые» сокращения

```
[2] import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt
```

Библиотека NumPy

Библиотека Numpy

- Предназначена для эффективной работы с многомерными векторами, матрицами и другими математическими объектами
- Очень быстрая. Поскольку написана не вполне на чистом Python

Библиотека Numpy

- Предназначена для эффективной работы с многомерными векторами, матрицами и другими математическими объектами
- Очень быстрая. Поскольку написана не вполне на чистом Python
- Главным объектом является Numpy Array
- Numpy Array обладает ключевой особенностью Array из c-подобных языков: может хранить объекты лишь одного типа (очень быстрый засчет этого)
- При этом может быть также динамически расширяем

Numpy Array

- Простой одномерный вектор

```
[2] vec = np.array([1, 2, 3])
```

```
vec
```

```
array([1, 2, 3])
```

```
[3] type(vec)
```

```
numpy.ndarray
```

Numpy Array

- Как вы думаете, а как задать матрицу?

Numpy Array

- Как вы думаете, а как задать матрицу?
- Массив массивов

```
[4] vec2 = np.array([[1, 2, 3], [4, 5, 6]])
```

```
vec2
```

```
array([[1, 2, 3],  
       [4, 5, 6]])
```

```
[5] type(vec2)
```

```
numpy.ndarray
```

Numpy Array

- Можно и трехмерный объект...

```
[9] vec3 = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

```
vec3
```

```
array([[[1, 2],  
        [3, 4]],  
       [[5, 6],  
        [7, 8]]])
```

```
[10] type(vec3)
```

```
numpy.ndarray
```

Размерности в Numpy

- Посмотреть размерность можно с помощью свойства ***shape***
- Какая размерность будет у `vec`, `vec2`, `vec3` из примеров выше?

```
[ ] print(vec.shape, vec2.shape, vec3.shape)
```

Размерности в Numpy

- Посмотреть размерность можно с помощью свойства ***shape***
- Какая размерность будет у `vec`, `vec2`, `vec3` из примеров выше?

```
[22] print(vec.shape, vec2.shape, vec3.shape)  
  
      (3,) (2, 3) (2, 2, 2)
```

Размерности в Numpy

- Также можно посмотреть количество осей с помощью свойства *ndim*

```
▶ print(vec.ndim, vec2.ndim, vec3.ndim, sep = " ")
```

```
☞ 1 2 3
```

Размерности в Numpy

- У некоторых функций бывает параметр `axis`, который позволяет применить эту функцию по разным осям - в данном случае, по строкам или столбцам
- Что выведет такой код?

```
[ ] np.sum(vec2)
```

```
[ ] np.sum(vec2, axis=0)
```

```
[ ] np.sum(vec2, axis=1)
```

```
[ ] vec2.sum()
```

Размерности в Numpy

```
[25] np.sum(vec2)
```

```
21
```

```
[26] np.sum(vec2, axis=0)
```

```
array([5, 7, 9])
```

```
[27] np.sum(vec2, axis=1)
```

```
array([ 6, 15])
```

```
[28] vec2.sum()
```

```
21
```

Размерности в Numpy

- Наконец, еще одна важная функция, связанная с размерностями – *reshape()*
- Позволяет менять размерность вектора, что иногда может быть очень полезно
- Что выведет этот код?

```
[ ] vec2.reshape(3, 2)
```

```
[ ] vec2.reshape(-1, 2)
```

```
[ ] vec2.reshape(3, -1)
```

Размерности в Numpy

```
[29] vec2
```

```
array([[1, 2, 3],  
       [4, 5, 6]])
```

```
[30] vec2.reshape(3, 2)
```

```
array([[1, 2],  
       [3, 4],  
       [5, 6]])
```

```
[31] vec2.reshape(-1, 2)
```

```
array([[1, 2],  
       [3, 4],  
       [5, 6]])
```

```
[32] vec2.reshape(3, -1)
```

```
array([[1, 2],  
       [3, 4],  
       [5, 6]])
```

Срезы в Numpy

- Работают очень похоже на обычные срезы в Python, но действуют по каждой оси (оси перечисляются через запятую)

```
[23] vec2 = vec2.reshape(3, 2)
      vec2

      array([[1, 2],
             [3, 4],
             [5, 6]])
```

```
[19] vec2[:, 1]
```

```
[20] vec2[2, :]
```

```
[21] vec2[1:2, 0]
```

```
[22] vec2[:, :2, :]
```

Срезы в Numpy

```
[23] vec2 = vec2.reshape(3, 2)  
vec2
```

```
array([[1, 2],  
       [3, 4],  
       [5, 6]])
```

```
[24] vec2[:, 1]
```

```
array([2, 4, 6])
```

```
[25] vec2[2, :]
```

```
array([5, 6])
```

```
[26] vec2[1:2, 0]
```

```
array([3])
```

```
[27] vec2[::2, :]
```

```
array([[1, 2],  
       [5, 6]])
```

Операции над матрицами

- Давайте вспомним линал! 😊
- Какие операции над матрицами вы помните?

Операции над матрицами

- Давайте вспомним линал! 😊
- Какие операции над матрицами вы помните?
- Сложение матриц
- Умножение матрицы на число
- Умножение матрицы на матрицу
- Транспонирование матрицы
- Могут быть и более сложные операции!

Операции над матрицами

- В NumPy реализована крайне удобная работа с матрицами, с точки зрения операций. Взгляните:

```
[28] vec2
      array([[1, 2],
            [3, 4],
            [5, 6]])

[29] vec2 + 10
      array([[11, 12],
            [13, 14],
            [15, 16]])

[30] vec2 + vec2 * (1/3)
      array([[1.33333333, 2.66666667],
            [4.          , 5.33333333],
            [6.66666667, 8.          ]])
```

Операции над матрицами

- И даже так:

```
[31] np.sin(vec2) + vec2 ** 2
```

```
array([[ 1.84147098,  4.90929743],  
       [ 9.14112001, 15.2431975 ],  
       [24.04107573, 35.7205845 ]])
```

Операции над матрицами

- И даже так:

```
[31] np.sin(vec2) + vec2 ** 2  
  
array([[ 1.84147098,  4.90929743],  
       [ 9.14112001, 15.2431975 ],  
       [24.04107573, 35.7205845 ]])
```

- То есть к матрице можно применить сразу, по сути, любую математическую функцию

Булевы массивы

- И еще один крайне важный функционал, появившийся в NumPy, – булевы массивы
- По сути, позволяют осуществлять полноценные фильтры данных, и очень понадобятся в дальнейшем

Булевы массивы

- И еще один крайне важный функционал, появившийся в NumPy, – булевы массивы
- По сути, позволяют осуществлять полноценные фильтры данных, и очень понадобятся в дальнейшем
- Что выведет такой код?

```
[ ] is_even = vec2 % 2 == 0  
    is_even
```

Булевы массивы

- И еще один крайне важный функционал, появившийся в NumPy, – булевы массивы
- По сути, позволяют осуществлять полноценные фильтры данных, и очень понадобятся в дальнейшем
- Что выведет такой код?

```
[33] is_even = vec2 % 2 == 0
      is_even

      array([[False,  True],
             [False,  True],
             [False,  True]])
```

Булевы массивы

- Еще пример

```
[34] new_vec = np.array([[1, 7], [8, 9], [-2, 0], [11, 3]])
```

```
[36] is_even = new_vec % 2 == 0  
is_even
```

```
array([[False, False],  
       [ True, False],  
       [ True,  True],  
       [False, False]])
```

Булевы массивы

- А теперь самая главная фишка
- Как вы думаете, что делает такой код?

```
[ ] new_vec[new_vec % 2 == 0]
```

Булевы массивы

- А теперь самая главная фишка
- Как вы думаете, что делает такой код?
- Та самая фильтрация данных

```
[34] new_vec = np.array([[1, 7], [8, 9], [-2, 0], [11, 3]])
```

```
[37] new_vec[new_vec % 2 == 0]
```

```
array([ 8, -2,  0])
```

Библиотека Pandas и Библиотека Matplotlib

Библиотека Pandas и Библиотека Matplotlib

- Numpy – это про числа и про математические объекты, а Pandas – это про данные
- Хотя подход к работе с таблицами почти точно такой же, как и в Numpy
- Matplotlib – это про графики и визуализацию

Библиотека Pandas и Библиотека Matplotlib

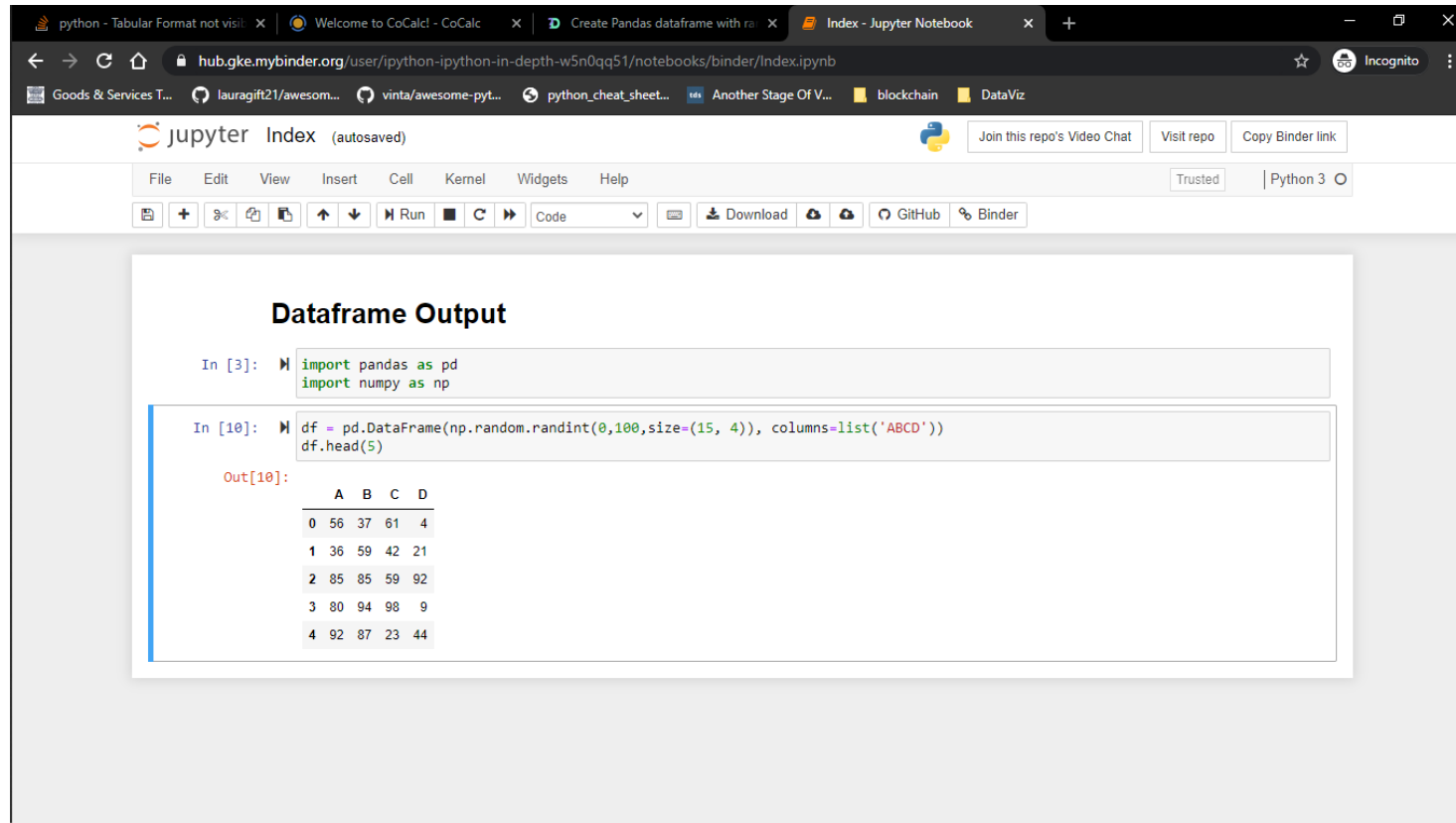
- Где мы будем с ними работать?

Библиотека Pandas и Библиотека Matplotlib

- Где мы будем с ними работать?
- Если для NumPy работать в классических питоновских средах программирования еще было возможно, то красиво визуализировать данные и тем более графики почти невозможно в стандартном консольном вводе-выводе

Jupyter Notebook и Google Colab

- Существует 2 рабочих варианта: Jupyter Notebook (Anaconda)



Dataframe Output

```
In [3]: import pandas as pd
import numpy as np

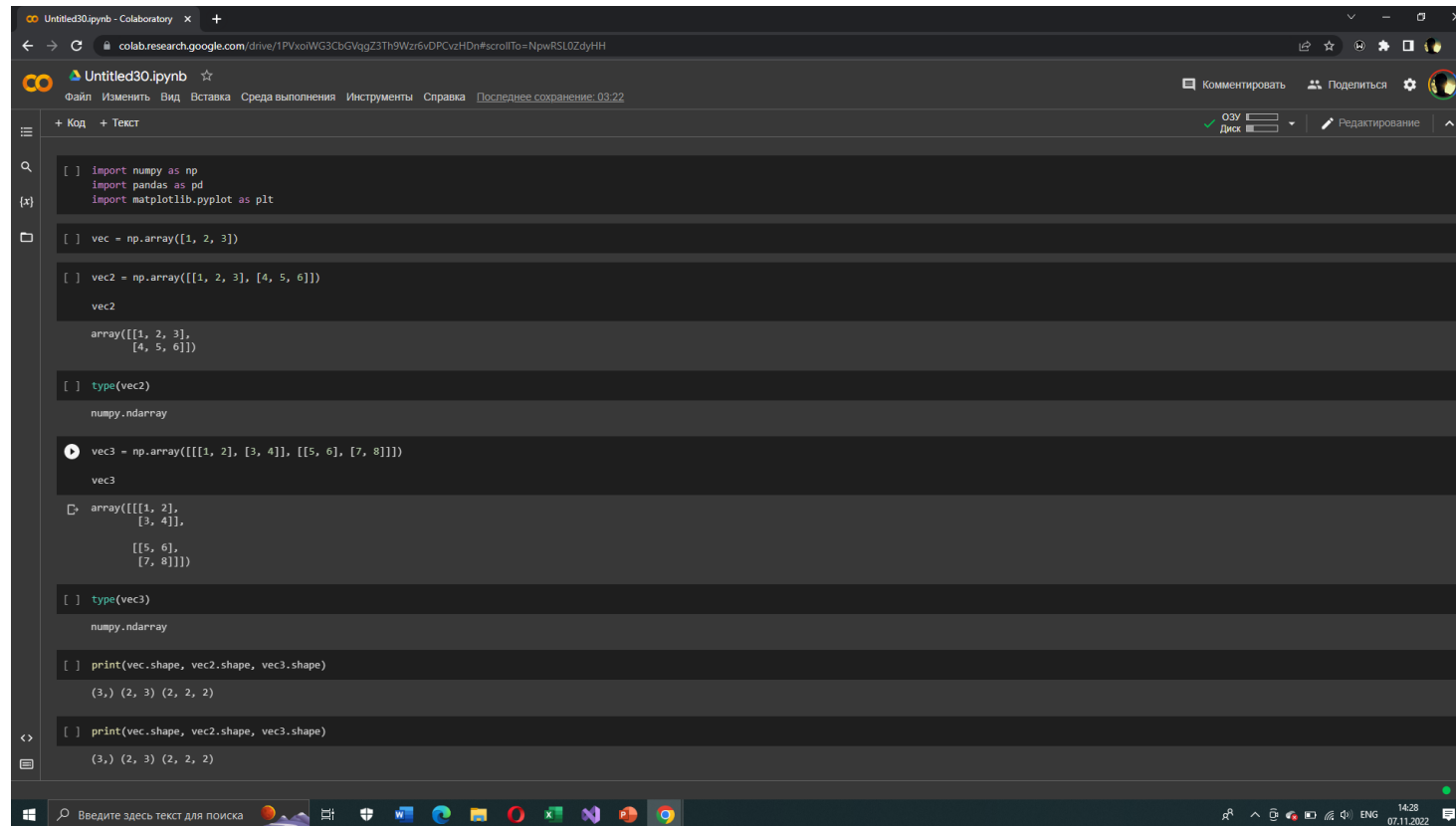
In [10]: df = pd.DataFrame(np.random.randint(0,100,size=(15, 4)), columns=list('ABCD'))
df.head(5)
```

Out[10]:

	A	B	C	D
0	56	37	61	4
1	36	59	42	21
2	85	85	59	92
3	80	94	98	9
4	92	87	23	44

Jupyter Notebook и Google Colab

- И бесплатная онлайн-среда от Google – Google Collaboratory



The screenshot displays a Google Colab Jupyter Notebook interface. The browser address bar shows the URL: `colab.research.google.com/drive/1PVxoiWG3CbGVqgZ3Th9Wz6vOPCvzHDn#scrollTo=NpwRSL0ZdytH`. The notebook title is "Untitled30.ipynb". The interface includes a menu bar with options like "Файл", "Изменить", "Вид", "Вставка", "Среда выполнения", "Инструменты", "Справка", and "Последнее сохранение: 03:22". On the right, there are buttons for "Комментировать", "Поделиться", and a settings icon. The left sidebar shows a file explorer with a search bar and a list of files. The main area contains a Jupyter Notebook with the following code cells:

```
[ ] import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

[ ] vec = np.array([1, 2, 3])

[ ] vec2 = np.array([[1, 2, 3], [4, 5, 6]])
vec2
array([[1, 2, 3],
       [4, 5, 6]])

[ ] type(vec2)
numpy.ndarray

[ ] vec3 = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
vec3
array([[[1, 2],
        [3, 4]],
       [[5, 6],
        [7, 8]]])

[ ] type(vec3)
numpy.ndarray

[ ] print(vec.shape, vec2.shape, vec3.shape)
(3,) (2, 3) (2, 2, 2)

[ ] print(vec.shape, vec2.shape, vec3.shape)
(3,) (2, 3) (2, 2, 2)
```

The bottom of the image shows a Windows taskbar with various application icons and a system clock indicating 14:28 on 07.11.2022.

Jupyter Notebook и Google Colab

- Подробнее с этими средами познакомимся на семинарах
- Они позволяют запоминать (на какое-то время) большое количество локальных переменных, перезапускать лишь некоторые части кода, а также многое другое
- В том числе, позволяют рисовать графики, писать текстовые комментарии, формулы, делать визуализацию

Данные в Pandas

- Скачаем данные из датасета о выживших на Титанике (возможно, самый известный датасет в сфере обучения анализу данных)

Данные в Pandas

```
[8] df = pd.read_csv("train (1).csv", sep=',')
```



df

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows x 12 columns

Основные типы в Pandas

- В Pandas существует два основных типа данных: Pandas DataFrame и Pandas Series
- Сами таблицы (в том числе и представленная на прошлом слайде) имеют тип Pandas DataFrame

```
[10] type(df)
```

```
pandas.core.frame.DataFrame
```

Основные типы в Pandas

- Pandas Series – это одномерный вектор среза таблицы
- Срез в виде столбца

```
[20] column = df['Name']  
      column  
  
0      Braund, Mr. Owen Harris  
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  
2      Heikkinen, Miss. Laina  
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)  
4      Allen, Mr. William Henry  
...  
886      Montvila, Rev. Juozas  
887      Graham, Miss. Margaret Edith  
888  Johnston, Miss. Catherine Helen "Carrie"  
889      Behr, Mr. Karl Howell  
890      Dooley, Mr. Patrick  
Name: Name, Length: 891, dtype: object
```

```
[19] type(column)
```

```
pandas.core.series.Series
```

Основные типы в Pandas

- Срез строки (менее частая история в Pandas)

```
[16] line = df.iloc[5]
line

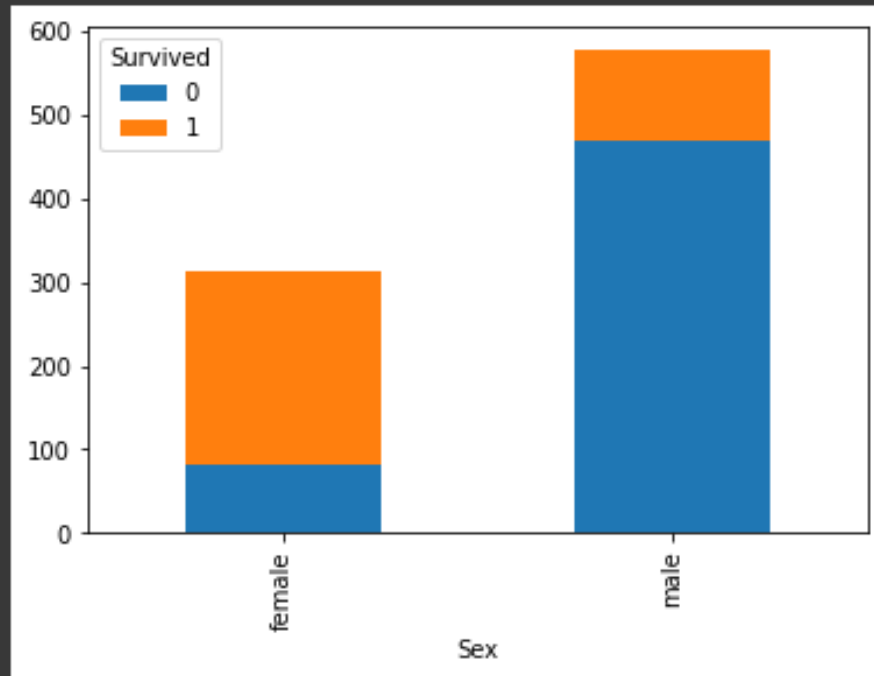
PassengerId      6
Survived         0
Pclass           3
Name      Moran, Mr. James
Sex            male
Age            NaN
SibSp           0
Parch           0
Ticket      330877
Fare         8.4583
Cabin          NaN
Embarked        Q
Name: 5, dtype: object
```

```
type(line)
```

```
pandas.core.series.Series
```

Немного визуализации под конец

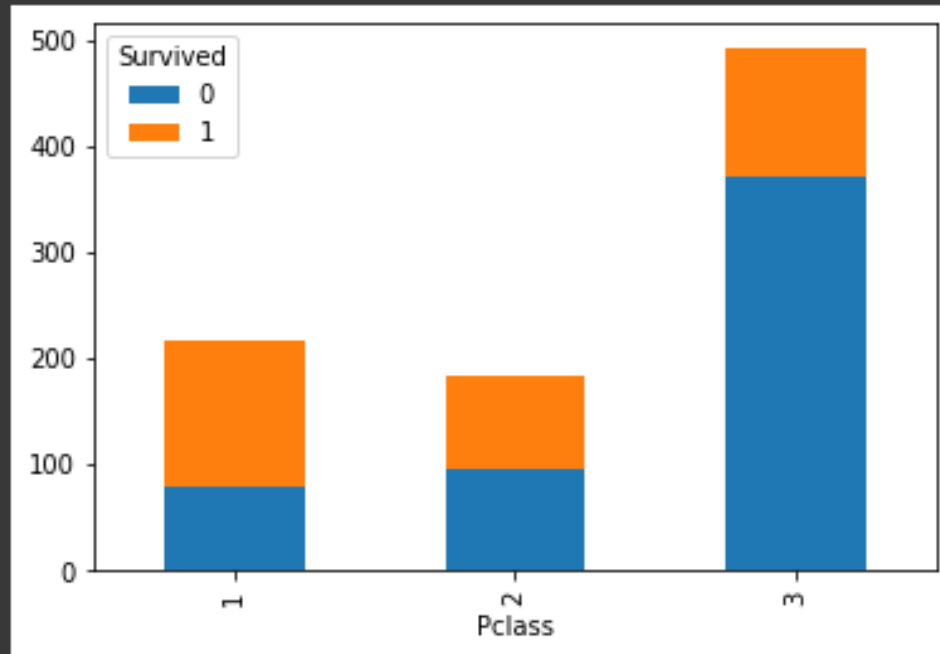
```
[22] df.pivot_table('PassengerId', 'Sex', 'Survived', 'count').plot(kind='bar', stacked=True);
```



Какие выводы
можно сделать?

Немного визуализации под конец

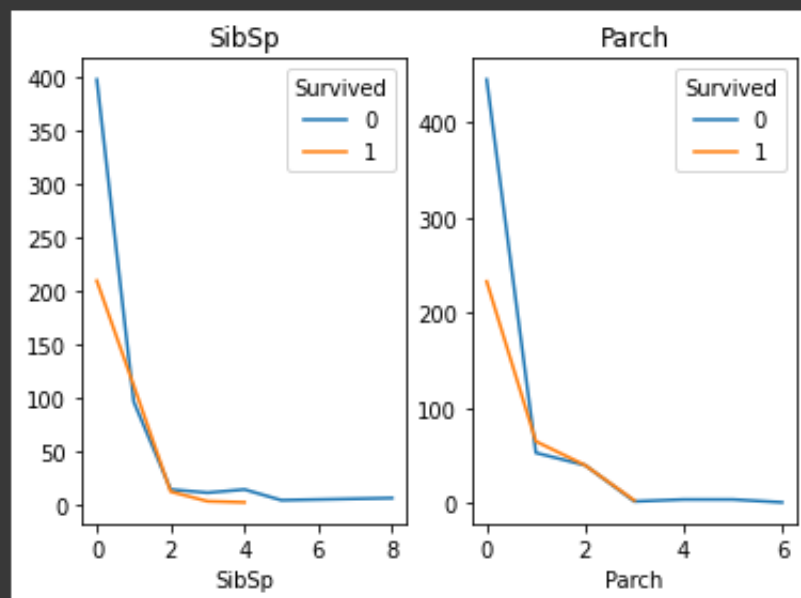
```
[23] df.pivot_table('PassengerId', 'Pclass', 'Survived', 'count').plot(kind='bar', stacked=True);
```



Какие выводы
можно сделать?

Немного визуализации под конец

```
[24] fig, axes = plt.subplots(ncols=2)
      df.pivot_table('PassengerId', ['SibSp'], 'Survived', 'count').plot(ax=axes[0], title='SibSp');
      df.pivot_table('PassengerId', ['Parch'], 'Survived', 'count').plot(ax=axes[1], title='Parch');
```



Какие выводы
можно сделать?